

2.º Examen parcial (R) – 11/12/2025

1. El *2-Partition Problem* como problema de optimización se describe tal que: Dado un conjunto de n números positivos $T = \{T_1, T_2, \dots, T_n\}$, se particionan los números en dos subconjuntos S_1 y S_2 (con intersección vacía y unión = T) de forma de minimizar la sumatoria de cualquiera de los subconjuntos ($\min \max(S_1, S_2)$).

Implementar un modelo de **programación lineal** que dados los valores de los T_i nos permita obtener la asignación óptima para S_1 y S_2 . Indicar la cantidad de inecuaciones definidas en el modelo.

2. Tito trabaja para la mafia. Debe llevar en camiones de caudales el dinero *recaudado* por la organización a modo de *devolver favores* a diferentes estaciones de policía. Entre diferentes rutas puede tener que cambiar de camión (para no ser fácilmente detectado), donde cada camión puede tener diferente capacidad. Además, en algunos puntos (escondites de la mafia) puede recoger más dinero.

Tito entonces cuenta con un grafo dirigido con las diferentes rutas, y puntos donde pueda tener que ir cambiando de camión según como cambie de ruta. El grafo es pesado y las aristas tienen como peso la capacidad del camión de caudales a usar por esa ruta. En cada vértice tenemos un punto donde podría (o no) poder recoger dinero o tener que dejar un *agradecimiento*, lo cual está modelado con un valor $w(v)$ (si es positivo, al pasar por allí podemos recoger dinero, si es negativo es que debemos dejar un *agradecimiento*, si es 0, es simplemente un punto de cambio de camión y nada más). Tito puede comenzar cada recorrido desde donde desee, si bien tiene sentido ir empezando desde algún escondite. Usando **redes de flujo**, implementar un algoritmo que reciba dicho grafo y determine si es posible pagar todos los *agradecimientos* correspondientes. No es necesario indicar cómo serían los recorridos. Indicar y justificar la complejidad del algoritmo implementado. El análisis de la complejidad debe ser completo.

3. El problema de *Coloreo de Boquita* se enuncia como: Dado un Grafo donde cada vértice tiene un peso $w(v)$ **positivo**, se desea pintar cada vértice de un color diferente de entre 4 posibilidades: Azul, Amarillo, Rojo o Blanco. Ningún vértice puede tener el mismo color de un adyacente. Aquellos vértices pintados con Rojo o Blanco nos dan como penalidad el peso de dicho vértice (los pintados de Azul o Amarillo no nos dan ninguna penalidad). Si excedemos un valor K en penalidad, nos vamos a la B. Dado un grafo y un valor K , ¿es posible pintar a los vértices, tal que ningún par de adyacentes compartan color (entre los 4 colores disponibles), y que la suma de las penalidades obtenidas sea a lo sumo K , de forma de no irnos a la B?

Demostrar que el problema de *Coloreo de Boquita* es un problema NP-Completo. Recomendamos recordar que 4-Coloreo es un problema NP-Completo.

4. Implementar un **algoritmo randomizado** que permita estimar el valor de π . Para esto, recomendamos recordar que el área de un cuadrado de lado 2 es 4, mientras que el área de un círculo de diámetro 2 es, casualmente, π . Explicar por qué el algoritmo implementado devuelve un valor aproximado de π .

2.º Examen parcial (R) – 11/12/2025

1. El *2-Partition Problem* como problema de optimización se describe tal que: Dado un conjunto de n números positivos $T = \{T_1, T_2, \dots, T_n\}$, se particionan los números en dos subconjuntos S_1 y S_2 (con intersección vacía y unión = T) de forma de minimizar la sumatoria de cualquiera de los subconjuntos ($\min \max(S_1, S_2)$).

Implementar un modelo de **programación lineal** que dados los valores de los T_i nos permita obtener la asignación óptima para S_1 y S_2 . Indicar la cantidad de inecuaciones definidas en el modelo.

2. Tito trabaja para la mafia. Debe llevar en camiones de caudales el dinero *recaudado* por la organización a modo de *devolver favores* a diferentes estaciones de policía. Entre diferentes rutas puede tener que cambiar de camión (para no ser fácilmente detectado), donde cada camión puede tener diferente capacidad. Además, en algunos puntos (escondites de la mafia) puede recoger más dinero.

Tito entonces cuenta con un grafo dirigido con las diferentes rutas, y puntos donde pueda tener que ir cambiando de camión según como cambie de ruta. El grafo es pesado y las aristas tienen como peso la capacidad del camión de caudales a usar por esa ruta. En cada vértice tenemos un punto donde podría (o no) poder recoger dinero o tener que dejar un *agradecimiento*, lo cual está modelado con un valor $w(v)$ (si es positivo, al pasar por allí podemos recoger dinero, si es negativo es que debemos dejar un *agradecimiento*, si es 0, es simplemente un punto de cambio de camión y nada más). Tito puede comenzar cada recorrido desde donde desee, si bien tiene sentido ir empezando desde algún escondite. Usando **redes de flujo**, implementar un algoritmo que reciba dicho grafo y determine si es posible pagar todos los *agradecimientos* correspondientes. No es necesario indicar cómo serían los recorridos. Indicar y justificar la complejidad del algoritmo implementado. El análisis de la complejidad debe ser completo.

3. El problema de *Coloreo de Boquita* se enuncia como: Dado un Grafo donde cada vértice tiene un peso $w(v)$ **positivo**, se desea pintar cada vértice de un color diferente de entre 4 posibilidades: Azul, Amarillo, Rojo o Blanco. Ningún vértice puede tener el mismo color de un adyacente. Aquellos vértices pintados con Rojo o Blanco nos dan como penalidad el peso de dicho vértice (los pintados de Azul o Amarillo no nos dan ninguna penalidad). Si excedemos un valor K en penalidad, nos vamos a la B. Dado un grafo y un valor K , ¿es posible pintar a los vértices, tal que ningún par de adyacentes compartan color (entre los 4 colores disponibles), y que la suma de las penalidades obtenidas sea a lo sumo K , de forma de no irnos a la B?

Demostrar que el problema de *Coloreo de Boquita* es un problema NP-Completo. Recomendamos recordar que 4-Coloreo es un problema NP-Completo.

4. Implementar un **algoritmo randomizado** que permita estimar el valor de π . Para esto, recomendamos recordar que el área de un cuadrado de lado 2 es 4, mientras que el área de un círculo de diámetro 2 es, casualmente, π . Explicar por qué el algoritmo implementado devuelve un valor aproximado de π .